locally defined users. When you log in, the password you type is compared with the encrypted and salted hash of your password in this file. As such, */etc/master.passwd* is vital to system security.

Editing /etc/master.passwd

If you're considering editing */etc/master.passwd* directly, ***stop***. Go back to Chapter 6. Reread it. See if there's another way to make your desired change. Damaging */etc/master.passwd* can prevent people from logging in at all, and might render your system unusable. To edit a single user account in */etc/master.passwd*, run `chpass(1)` as root.

If you must edit */etc/master.passwd* directly—say, to change everyone's home directory to point to a new filesystem—there's a program just for that. The program `vipw(8)` calls up the text editor in `$EDITOR`, loads the password file, lets you make changes, and checks the file syntax before saving it. `vipw` also updates */etc/passwd* and the password databases */etc/pwd.db* and */etc/spwd.db*.

Be absolutely sure that this file is synchronized with the password databases. Using `vipw` prevents many basic mistakes and helps ensure data consistency, but if you're really bent on corrupting */etc/master.passwd*, `vipw(8)` will make your task more difficult but won't stop you.

That said, only senior sysadmins should use `vipw`. How do you know if you qualify? If you've made enough horrible mistakes with `vipw` that you know in your bones not to use it, if your stomach churns at the mere thought of typing those four letters, you may use `vipw`.

Controlling Account Information Access

Many programs need parts of the information stored in */etc/master.passwd*. For example, a program must be able to look up a user's shell and home directory to properly find the user's files. Rather than allowing anyone to read this file and the scrambled passwords therein, OpenBSD (and most other Unix-like systems) provide the account information everyone needs to see in */etc/passwd*.

Accessing and parsing a text file can be slow, especially if the computer has a slow processor and many user accounts. Text files are not meant for searching. If a program must search a text file for user account number 10631, the search can hold up the program's activity and even block other programs. Checking a database for an entry is much faster, as the program can just say "Give me account 10631" and get a response from a file store intended for searches.

All Unix-like systems create databases from the password files whenever the account information changes. The database of public account information, built from */etc/passwd*, is */etc/pwd.db*. The database file */etc/spwd.db* contains private account information built from */etc/master.passwd*.

Realistically, very few pieces of software actually use the password file directly. Most programs access *pwd.db*. I know people who delete */etc/passwd* without ill effect, but you do need to keep the password database.

/etc/master.passwd Fields

Each account is a line in */etc/master.passwd* and */etc/passwd*. Each line has the following nine fields, separated by colons.

Username
: This field contains either an account created by the sysadmin and used by a human, or a nonprivileged user created for use by a program or service. Chapter 6 covers usernames.

Hashed and Salted Password
: This field contains the user's password, hashed and salted. (It's commonly called a hash, but you should know it's more than that.) You cannot derive the password from the hash, but if you have the hash, you can try passwords until you find one that matches the hash. That's why you must protect your */etc/master.passwd* file. In */etc/passwd*, the password field is blank.

NOTE *One simple way to temporarily disable a user account is to edit the password file with `chpass(1)` and put an asterisk (*) in front of the hash. While the account will still be active, no one will be able to log in to it. I've used this to great effect when a client is behind on a bill. While clients can ignore overdue payment notices, they call quite quickly when they cannot access their account. I can easily route that call to the billing department. Once the matter is resolved, I can re-enable their account by removing the asterisk.*

User ID Number
: The third field is the user ID number, or UID. Every user must have a unique UID.

Group ID Number
: This field is the group ID number, or GID. This is the user's primary group, as discussed in Chapter 6. Usually, this is the same as the UID, and the user's primary group has the same name as the username. Some sites prefer to use a single group for all unprivileged users.

User Class
: We discuss the user class in Chapter 6. Changing the class can increase or decrease the amount of system resources the user can access.

Password Expiration Date
: The expiration date is expressed in the number of seconds since midnight, January 1, 1970, the UNIX epoch. You can convert dates to seconds with `date(1)`. If you must manually set an expiration date for a password, use `chpass(1)` and specify a human-readable date.

Gecos
: This field contains the user's real name, office number, work phone number, and home phone number, all separated by commas. This information was much more important when computers were big systems with hundreds or thousands of users, and you might need to contact people when their process went completely insane. Today, it's basically ignored. (Do not use colons here; colons are reserved specifically for separating fields in */etc/master.passwd*.)

User's Home Directory
: This field is the user's home directory. While this defaults to a directory with the same name as the username, beneath */home*, you can move this directory anywhere you like. Editing the home directory in the password file does not move the actual directory; you need to do that yourself.

User's Shell
: The last field gives the user's shell. If this field is empty, the user gets boring old */bin/sh*.

/etc/mixerctl.conf

OpenBSD includes solid audio abilities. You can listen to MP3s, mix music, or just about anything else you would like.

Control audio settings with `mixectl(8)`, and set boot-time `mixerctl` settings in */etc/mixerctl.conf*.

/etc/mk.conf

Configure `make(1)` with */etc/mk.conf*. The most common use for special `make` settings is while you are building ports (covered in Chapter 13) or while building your own custom OpenBSD release (Chapter 20).

/etc/moduli

The */etc/moduli* file contains prime numbers, used for Diffie-Hellman cryptography. Never edit this file. Some people may understand cryptography well enough to edit */etc/moduli*, but if you're reading this book, you're probably not one of them.[2]

/etc/monthly and /etc/monthly.local

The */etc/monthly.local* shell script runs once per month as part of routine system maintenance. Chapter 15 discusses scheduled maintenance jobs.

2. Henning Brauer, OpenBSD developer and point man on packet filtering, comments, "I understand crypto somewhat well. . . . I don't muck with moduli either tho." Consider yourself warned.

/etc/motd

The MOTD, or message of the day, is displayed to users upon login. In this file, you might put system notices or announcements that you hope users will notice. Many organizations put legal notices or acceptable use policies in */etc/motd*.

Note that the first line of */etc/motd* is overwritten at every boot. Start your legal warning in line 2 or below.

/etc/mrouted.conf

In addition to `dvmrpd(8)`, OpenBSD supports multicast routing with `mrouted(8)`. The `dvmrpd` implementation is preferred, but for specific edge cases, you might need `mrouted` instead.

Configure `mrouted` in */etc/mrouted.conf*. Eventually, `mrouted` will be removed from OpenBSD.

/etc/mtree/

The */etc/mtree/* directory contains a list of most directories on a stock OpenBSD system, with their ownership and permissions. The system upgrade process uses this. While you don't really need to edit these files, it's nice to know what the heck they're for.

/etc/mygate

The */etc/mygate* script gives the address of the default gateway for both IPv4 and IPv6, as discussed in Chapter 12.

/etc/myname

The */etc/myname* file contains the hostname of the system, as discussed in Chapter 4.

/etc/netstart

The */etc/netstart* script starts the network, as discussed in Chapter 12.

/etc/networks

/etc/networks is a list of subnets and their names. Use of a networks database has fallen out of favor, because it's not terribly useful.

/etc/newsyslog.conf

The `newsyslog(8)` program rotates log files, as discussed in Chapter 15.

/etc/nginx/

OpenBSD has imported the `nginx` web server (*http://www.nginx.org/*) as a replacement for the older Apache 1.3 server, but as of this writing, it's not

quite integrated with the rest of the system. You can find the `nginx` configuration files in this directory, and the server is quite usable, but it's not the official OpenBSD web server—yet.

/etc/nsd.conf

OpenBSD has imported the name server daemon `nsd(8)` to eventually partially replace the old DNS workhorse server `named(8)`. It's usable, but as of this writing, it's not yet integrated with the system.

/etc/ntpd.conf

The NTP daemon keeps the system time synchronized with other machines on your network and the Internet. We discuss `ntpd(8)` in Chapter 15.

/etc/ospf6d.conf and /etc/ospfd.conf

OSPF is a routing protocol used inside autonomous networks. OpenBSD has two OSPF implementations: one for IPv4 and one for IPv6. If you want to know more about OSPF, read `ospfd(8)` and `ospf6d(8)`.

/etc/pf.conf and /etc/pf.os

Configure the OpenBSD `pf(4)` packet filter in */etc/pf.conf*. Packet filtering uses */etc/pf.os* to fingerprint other operating systems. Chapters 21 and 22 cover packet filtering.

/etc/ppp/

You can connect OpenBSD to the Internet via a dial-up modem, which is rarely done these days. If you need to configure a modem on your OpenBSD system, read the `ppp(8)` man page.

/etc/printcap

The printer capability file describes all printers that this system can access. Making a Unix-like system work with a random printer was long considered to require some sort of sacrifice, a moon in the correct phase, and a team of chanting acolytes in robes. While complicated software such as CUPS has been written to simplify printing, configuring an OpenBSD machine to print to a print server or a network PostScript printer is pretty simple. See Chapter 16 for details on printing.

/etc/protocols

The */etc/protocols* file contains protocol numbers for TCP/IP network protocols. Chapter 11 covers TCP/IP versions 4 and 6 in detail.

/etc/rbootd.conf

`rbootd(8)` offers boot services for HP workstations—a very narrow subset of obsolete diskless clients. OpenBSD still supports the HP300 machines that need this service. If you're interested in diskless operations on modern hardware, read the `diskless(8)` man page instead, or look at Chapter 23.

/etc/rc.*

The */etc/rc.** files are used for system booting, as discussed in tortuous detail in Chapter 5.

/etc/relayd.conf

The load balancer daemon `relayd(8)` works with the OpenBSD Packet Filter (PF) system to act as a network load balancer. The `relayd` daemon requires a good understanding of PF, however, and a very specific network. If you're interested in load balancing, read Hansteen's *The Book of PF*.

/etc/remote

Unix-like systems have extensive support for connecting into the system over serial lines, usually for serial consoles. Many network appliances have management serial ports, and you can use OpenBSD as a client to configure these devices. The */etc/remote* file contains serial connection configurations for most common modern serial connections (covered in Chapter 5).

/etc/resolv.conf and /etc/resolv.conf.tail

The */etc/resolv.conf* and */etc/resolv.conf.tail* files configure the resolver (covered in Chapter 12), letting the host map names to IP addresses and vice versa.

/etc/ripd.conf

RIP is an old way to broadcast routing instructions across a network. OpenBSD has a RIP daemon, `ripd(8)`, configured in */etc/ripd.conf*.

RIP is generally considered undesirable, like the r-services. Among other shortcomings, it doesn't even support netmasks, so it's restricted to old-style class A, B, and C networks. And as with the r-services, sometimes you're stuck with RIP because some obsolete device on your network can handle nothing else. Use `ripd` to scrape by until you can arrange a tragic accident for that device.

/etc/rmt

The remote magnetic tape command (`rmt`) lets a host access a tape drive on another machine. It's most commonly used to restore a system from backup.

/etc/rpc

RPC is a method for executing commands on a remote server. Much like TCP/IP, RPC has service and port numbers. The file */etc/rpc* contains a list of these services and port numbers. The most common RPC consumer in OpenBSD is NFS, as discussed in Chapter 9.

/etc/sasyncd.conf

OpenBSD supports failover between IPsec gateways, using the security association synchronization daemon `sasyncd(8)`. This is not a common feature in operating systems, and its presence is a highlight in OpenBSD. To learn about IPsec failover, read `sasyncd(8)`.

/etc/sensorsd.conf

Modern hardware has sensors for detecting items like fan speed, circuit voltage, temperature, and so on. OpenBSD's `sensord(8)` reads these sensors and presents the information to the user. Configure which sensors you want to pay attention to, as well as what you want to do when the sensors detect something, in */etc/sensorsd.conf*. See Chapter 15 for details on `sensorsd`.

/etc/services

The */etc/services* file contains a list of network services and their associated TCP/IP ports. See Chapter 11 for details.

/etc/shells

The */etc/shells* file contains a list of legitimate user shells, as discussed in Chapter 6.

/etc/skel/

The */etc/skel/* directory contains standard user environment configuration files. When you create a user account, `adduser(8)` copies the files in this directory to the new user's home directory. This directory can be overwritten when you upgrade your system.

If you need to customize these files for your users, create a custom dotfile directory and tell `adduser(8)` to use it instead.

/etc/sliphome/

The Serial Line Internet Protocol (SLIP) predated the Point-to-Point Protocol (PPP) commonly used for dial-up lines. OpenBSD still supports it, as someone might need it and there's no real reason to get rid of it.

/etc/snmpd.conf

Simple Network Management Protocol (SNMP) is a method for accessing information about a device over the network. Unfortunately, it's not a terribly secure protocol (one common acronym for SNMP is "Security? Not My Problem!") The OpenBSD team has written a more secure SNMP daemon, `snmpd(8)`. Configure it in */etc/snmpd.conf*, as discussed in Chapter 16.

While OpenBSD's SNMP daemon might resist intrusions and abuse, it can't help the fact that SNMP itself, as commonly deployed, isn't terribly secure.

/etc/ssh/

The SSH daemon `sshd(8)` offers a secure replacement for `telnet(1)` and the r-protocols. Chapter 16 includes a brief discussion of SSH.

/etc/ssl/

The */etc/ssl/* directory is for Secure Sockets Layer (SSL) certificates, as well as the OpenSSL configuration file *openssl.cnf*. Store system SSL certificates here.

/etc/sudoers

The */etc/sudoers* file controls `sudo(1)` configuration. See Chapter 7 for details about `sudo`.

/etc/sysctl.conf

Set kernel runtime tunable options in */etc/sysctl.conf*. Sysctls are covered in Chapter 18.

/etc/syslog.conf

The logging daemon `syslogd(8)` reads messages from programs and hosts, and then separates those messages into records based on the configuration in */etc/syslog.conf*. See Chapter 15 for details on `syslogd`.

/etc/systrace/

The `systrace(4)` system call wrapper provides access controls to system calls. You could run a binary "under" `systrace(1)`, and if the program attempted to access any system call beyond those permitted in the application's `systrace` policy, `systrace` would block the access.

Flaws were found in `systrace`, however, that make it less than effective, and it's now considered only a partial solution. It still ships with OpenBSD, but its use is discouraged. Today, `systrace` is mostly used for package-building clusters to make sure that software built doesn't phone home or write outside the fake installation root.

If you need to use `systrace`, store policies in */etc/systrace*.

/etc/termcap

The */etc/termcap* file describes all the different terminals that OpenBSD supports. Pretty much every console device now supports the standard VT220 terminal, however.

/etc/ttys

Configure system terminals in */etc/ttys*. You can enable, disable, and change terminals here. A "terminal" could be the keyboard and monitor attached to the computer, a login over a serial line as with a serial console, or a virtual terminal as used by telnet or SSH.

A classic UNIX terminal device resembled a teletype; that's where the *tty* label comes from. All sorts of UNIX architectural details descend from this historical accident, and Unix-like systems inherited them.

Terminal Types

/etc/ttys lists three different terminal types: the console, serial ports, and pseudo-terminals.

The console is where boot messages display, where single-user mode maintenance can happen, and where error messages directed at the console appear. While the console is usually the keyboard, monitor, and mouse attached to the computer, it doesn't need to be that. The "console" is an abstraction that usually happens to be aimed at your physical monitor and keyboard. You can direct the console at a serial port instead, for example. The console uses the device */dev/console*.

On some platforms, including i386 and amd64, OpenBSD supports multiple virtual consoles attached to your physical console. If you press CTRL-ALT-F2 on the physical keyboard, you'll see a brand-new login screen. CTRL-ALT-F1 returns you to the main console. You can have as many virtual consoles as you have function keys, but OpenBSD has five by default. These virtual consoles have device names beginning with */dev/ttyC* and ending in a hexadecimal number.

Serial ports can be used as login devices, once you attach either an old-fashioned serial terminal or a null-modem cable and another device with a serial port. They can also be used for outgoing serial connections. Serial ports used as incoming devices start with */dev/tty*, while ports used for outgoing connections start with */dev/cua*, and both end in hexadecimal numbers. Each serial port can have one terminal attached to it.

Pseudo-terminals support network connections. Even though your remote SSH window has no corresponding physical hardware on the server, OpenBSD treats it in some way as a `tty` device. Pseudo-terminals have device names beginning with */dev/tty*, a letter `p` through `z` (either lowercase or uppercase), and ending in a single letter or number. When a user connects over the network, such as via SSH, the login session is assigned one of these virtual terminal devices.

Whatever the terminal type is, configure it in */etc/ttys*, using the following procedure.

Configuring Terminals

Each terminal has an */etc/ttys* entry, containing at least three entries and possibly up to five.

```
ttyC0 /usr/libexec/getty std.9600 vt220 on secure
```

The first entry is the device name, without the leading */dev*.

The second entry is the name of the program that spawns login requests on the terminal. Physical terminals and virtual consoles use `getty(8)`, while pseudo-terminals process login requests through whatever daemon the user logs in through.

The third entry is the terminal type. OpenBSD uses a `vt220` terminal on the monitor and the virtual consoles. Serial consoles use the `unknown` terminal type, as there's no way to know in advance what kind of hardware is on the other end of the terminal. (You can switch this to `vt220` without issue if you're using a reasonably modern serial client.) Pseudo-terminals use the `network` terminal type; the server daemon and client software determine the terminal's features.

The fourth field determines if the terminal is available for logins. Set this to `on` to accept login requests, or `off` to deny them. Pseudo-terminals are activated on demand, so you don't need to configure them in */etc/ttys*.

The root account can log on to only a secure console. The physical terminal and the console are the only devices defined as secure by default. You can log in as a regular user and use `su(1)` or `sudo(1)` on an insecure terminal; you just can't directly log on as straight `root`. This means that anyone who has the root password and physical access to the machine can just walk up to the keyboard and log in as root. To disallow logins directly to root, remove the `secure` keyword from the virtual console entries in */etc/ttys*. Also, on secure terminals, you won't be asked for the root password when you boot into single-user mode.

If you use serial consoles, you might want to log in to a running multiuser system over the serial cable. By default, you cannot log in on a serial line. Set the serial line with the serial console to `on`, and you'll be able to log in. This makes a serial line behave much like being at the physical console, where you can work in multiuser mode and while the machine is running normally.

While all of the defaults use `getty(8)`, there are alternatives. For example, the HylaFAX package lets you connect a fax machine to a serial line, but you need to reconfigure the terminal to support it.

```
ttyOe   "/usr/local/sbin/faxgetty"      dialup  on
```

The point is that you can use serial ports any way you like, once you get the right software tool for the job.

Making /etc/ttys Changes Take Effect

Offering terminals is a low-level system task handled directly by `init(8)`. Changes to */etc/ttys* do not take effect until you tell `init` to reread its configuration. (`init` is always process ID 1.)

```
# kill -HUP 1
```

If you don't tell `init` to reread its configuration, changes won't take effect until you reboot.

/etc/weekly and /etc/weekly.local

The */etc/weekly.local* script runs once a week to perform weekly maintenance, as discussed in Chapter 15.

/etc/wsconsctl.conf

OpenBSD includes hardware-independent access to the physical console through the `wscons(4)` driver. Configure this console via `wsconsctl(8)`. The boot-time `wsconsctl` settings are read from */etc/wsconsctl.conf*. See Chapter 17 for details on console configuration.

/etc/X11

The */etc/X11* directory contains configuration for the X Window System. OpenBSD's Xenocara integrates X with the base system. Chapter 17 discusses desktop OpenBSD, including some X features.

/etc/ypldap.conf

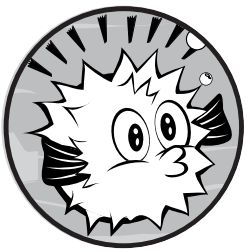
In addition to the LDAP daemon `ldapd(8)`, OpenBSD supports the YP database for centralizing passwords, groups, and host filesystems. YP is compatible with Sun's original Network Information System (NIS). OpenBSD uses YP as a gateway to LDAP authentication. If you're interested in this feature, see `yp(8)` and `ypldap(8)`.

This takes you through everything in */etc* not covered elsewhere. In the next chapter, we'll discuss how OpenBSD maintains itself, and how you can meddle with the maintenance processes.

15

SYSTEM MAINTENANCE

When hardware complains,
OpenBSD listens.
You should listen too.



No computer runs itself. If it did, you would be out of a job. Even the best-configured server generates a constant low burn of maintenance needs.

OpenBSD includes a variety of tools to make maintenance easier, to alert you when maintenance is needed, and to tell you about the system's status. Every day, week, and month, OpenBSD performs maintenance tasks and notifies you of the results. OpenBSD takes daily backups of critical system features and files, and uses them to monitor system integrity. It can manage its own log files, keep its own time, and alert you when the hardware is failing.

All of this starts with scheduled maintenance.

Scheduled Tasks

OpenBSD includes scheduled tasks that run once a day, week, and month. These jobs run as root and email the results to root. The daily maintenance script is the most complicated; the monthly script is the simplest.

If a server runs well, I might not log in to it for weeks or even months. In fact, I've had a few servers run for more than a year without a human being ever logging in to it. I read the daily reports from the machine and say "Yep, it's okay," and get on with my day, confident that what the regular status reports don't tell me the monitoring system will.

The scheduled maintenance jobs email their results to the local root user, but if no one ever logs in to the machine, no one will see those results. Always set an email alias for root in */etc/aliases*, so that these messages go to someone who will actually read them.

Reading every email message from every machine every day is annoying, but much less annoying than finding out that I have a bad service by a user telling me about it. These messages often alert me to system problems before anyone (including me) notices them. Sites with hundreds of machines often write scripts to parse incoming email messages and flag interesting details.

You should send maintenance email to the person ultimately responsible for the system—whoever is most interested in system changes and most likely to be aware of any day-to-day system changes. If you delegate the job of reading maintenance email to a minion who is less aware of the system, he will either annoy you with endless questions about what you did yesterday or learn to ignore anything actually in the status mail messages.

Here, we'll take a look at how the daily, weekly, and monthly routines work. Complete documentation of the maintenance jobs appears in `daily(8)`.

Daily Maintenance

The daily maintenance job starts by running any custom maintenance jobs (covered at the end of this section) from */etc/daily.local*. Daily maintenance includes checking partitions, running the reminder service `calendar(1)`, running `rdist(1)`, removing scratch files, and a few other boring things.

OpenBSD can also do a few other interesting things as part of its daily maintenance:

- Create a backup root filesystem, */altroot*.
- Perform system security checks.
- Back up vital system files in */var/backup*.
- Check for changes in vital system files.
- Check filesystem integrity.
- Run `rdist(1)`.

I discussed */altroot* in Chapter 9 because it requires a dedicated filesystem partition. Each of the other tasks can be configured later.

Security Checks

Some things that go wrong don't necessarily mean your system has experienced an intrusion, but are nonetheless suspicious. The daily security

check looks for a whole slew of misconfigurations and problems that arise from either malice or incompetence. You can read the list of checks in `security(8)`, but they break down into fairly broad categories:

Device node changes and privileges
: New device nodes, changed permissions on device nodes, new software packages, and new or altered disks or partitions might indicate malicious activity or might just be normal system management. The security script flags all of these. If you made such changes, you'll nod and go on with your day. If you requested a minion perform the change and the change doesn't appear, this is when you ask them what they did all day yesterday. If you didn't make a change that appears, you want to know about it.

Insecure NFS exports
: OpenBSD includes a lot of software to export filesystems and run commands remotely. These services, like printing and NFS, should not allow access from any host just anywhere, but only from hosts you approve. The security job checks for configurations that permit global access.

Misconfigured accounts
: Another popular attack route has been the password database and related files. Accounts without passwords, duplicate entries, improperly closed accounts, and so on could all be used to compromise a system. The script checks for these issues.

Permissions
: Poor permissions can lead to privilege escalation. For example, is there a new `setuid` or `setgid` file on the system? If so, the security script notifies you. If you installed that file with those privileges, you're okay. If it's unexpected, you should investigate.

User environment
: If you can change a user's environment, whether that's root or another account, you can trick him into giving away his authentication credentials or running suspect commands. If an intruder can edit a user's dotfiles, like *.cshrc* or *.login*, he can change which versions of a command he runs. Perhaps his shell is set to run a program that asks the user for his password and sends it to the intruder's anonymous email account. By having correct permissions on home directories, dotfiles, mail files, and so on, you make this class of attack more difficult. The security script verifies that permissions are set up correctly.

Note that the security check is *not* an intrusion-detection system. The changes it checks for are the sort that script kiddies and newbie intruders are most likely to make, but skilled intruders familiar with OpenBSD could get around it. They could even replace the security check with a shell script that sends a daily email message that looks like an innocuous security check.

Fortunately, competent intruders are relatively rare. Just keep in mind that receiving a security check with no mention of problems is encouraging, but it's not proof that your server is secure.

Vital File Backup and Testing

The daily security check tests for changes in the files listed in */etc/changelist* and rotates their backups, since these files are generally critical system files, such as */etc/master.passwd*, */etc/boot.conf*, and */var/cron/tabs/root*. It also checks for changes to disk partitioning and mounted filesystems, as well as changes to device nodes.

Look in */var/backups*, and you'll see files like this:

```
...
etc_fstab.backup
etc_fstab.current
etc_ftpchroot.current
etc_ftpusers.current
...
```

The files ending in *.current* are copies of these files as they existed when the daily maintenance job was last run. The files ending in *.backup* are the previous version of those files.

The first time the security script runs, it copies all of these files to */var/backup*. Following that initial setup, the security script checks the original file against the current copy for changes. If the file changes, the previous version of the file is copied to the *.backup* filename, and the new version is copied to the *.current* file.

In the preceding example, the list shows that I edited my */etc/fstab* at some time, prompting the security script to move its copy of the old filesystem table to a *.backup* file. I have never edited */etc/ftpchroot* or */etc/ftpusers*, so there is no *.backup* version of these files, but only the *.current* one.

The security script doesn't copy all of the files that it watches. For example, files containing private keys or that might contain private keys are not copied, but the security script does take a checksum. (Files monitored by checksum have a plus sign before their name in */etc/changelist*.) There's no reason to manually edit */etc/ssh/ssh_host_ecdsa_key*, and if the file changes, either you know why or you need to restore from a trusted backup.

/etc/changelist is itself listed in */etc/changelist*. This seems recursive, but the system backs up the list of files you want backed up, and also notifies you when someone adds or removes a file in */etc/changelist*.

Adding Vital Files

You can add files to the change list and even use wildcards to back up all the files in a directory. But note that if you did use wildcards in */etc/changelist*, you won't be notified when a file is removed.

Consider this example of using wildcards. In Chapter 13, I added the `apache2` port to one of my machines. I put the configuration files in */etc/apache2*. I could add a line like this to the change list:

```
/etc/apache2/*
```

This would automatically copy all files in the *apache2* configuration directory to */var/backup* and test them for changes. However, I would not be notified when files are removed from this directory. If you're using a configuration mechanism that says, "include all the *.conf* files in such-and-such directory," this might not be desirable. A better option would be to list each file individually and update the list when you add critical files.

One of the most convenient things about the file-integrity check is that it automatically creates a local backup of critical system files. That means that if you decide to learn how to use `vipw(8)` and utterly trash your user database in doing so, you can grab yesterday's copy out of */var/backups*, install it, and no one will be the wiser. The same applies to every other critical system file.

Filesystem Integrity Checks

You can't run a full-on Unix File System (UFS) check while a system is in multiuser mode, but you can have `fsck(8)` perform filesystem integrity checks to try to identify problems before they're serious. Doing so won't fix any problems, but it will notify you that they exist so you can schedule downtime for repairs.

To enable these checks, set `CHECKFILESYSTEMS` to `1` in */etc/daily.local*.

Copying Files with rdist

The `rdist(1)` program is used to copy files to other servers, letting you maintain identical copies of critical files on many servers. If you're interested in using it, see `rdistd(8)`.

Silencing /etc/daily

Some of us have monitoring systems that track a server's disk, network, and other basic information. If you don't need this sort of information to appear in your daily status mail, set `VERBOSESTATUS` to `0` in */etc/daily.local*. This turns off these parts of daily maintenance, reducing the amount you need to read.

If the remaining daily maintenance doesn't generate any output, the server should not send a status email that day. In environments where you don't trust the monitoring system, you could use the daily status messages to assure you that the system is running as expected. OpenBSD gives you the choice.

Weekly Maintenance

The weekly script is simpler than the daily script with only three common functions:

- First, it runs the custom weekly script */etc/weekly.local*.
- Second, it updates the `locate(1)` database.
- Finally, it rebuilds the `whatis(1)` man page database.

Monthly Maintenance

OpenBSD doesn't need any generic monthly maintenance, but for consistency, the */etc/monthly* script runs the custom script */etc/monthly.local*.

Custom Maintenance Scripts

Each maintenance script runs a custom script before performing any other tasks. You can put any tasks you need in */etc/daily.local*, */etc/weekly.local*, and */etc/monthly.local*. These commands are run by root, so don't use them for tasks that should be performed by another user. If your database needs to be backed up, create a separate script, and have the unprivileged user running your database run that script via `cron(8)`.

Some sites use the scheduled maintenance jobs to run complex software that perform site-specific duties. For example, I know of one security firm that collects data from hundreds of machines, and uses the daily jobs to send that data to a central management system. Really, you can use the local scripts any way you choose.

If you have a maintenance task that can run under another user account, but you want to attach it to the scheduled maintenance jobs, you can have the local script call another script. Start that script by using `su(1)` to switch users and drop privileges.

Custom maintenance scripts may be most useful for altering the way the standard maintenance scripts perform their work. For example, say you have a system with many scratch directories containing temporary files. The weekly maintenance script updates the locate database, but you don't want these scratch files included in locate results. You could use a custom maintenance script to remove all the scratch files immediately before */etc/weekly* creates the new locate database, and schedule this as a separate task. By adding it to */etc/weekly.local*, you would know that it will finish before */etc/weekly* runs any other tasks.

System Logs

The system log used by Unix-like operating systems has become the industry standard for logging, but that's not necessarily a good thing, because the log mechanism can be cantankerous. Once you properly configure log collection and rotation, however, OpenBSD's logging system mostly manages itself.

OpenBSD uses the standard logging system for Unix-like (and many embedded) systems, `syslog(3)`. The syslog protocol marks messages with a facility and a priority, and hands those messages to a daemon.

Any program can write to the local `syslogd(8)` server, but the key in log management is deciding how those messages are sorted and stored. OpenBSD's `syslogd` can sort messages based on facility, priority, and source program.

Facilities

A *facility* indicates the source of a message. In most cases, each program that needs a separate log file uses a different facility. Many programs or protocols, such as FTP, have facilities dedicated to them. The syslog protocol also has a variety of generic facilities that you can use as you wish.

Table 15-1 lists the standard facilities and provides some notes on their usage.

Table 15-1: Standard OpenBSD Facilities

Facility	Usage
`auth`	Public information about authentication, such as when someone logged on or when someone uses `su`.
`authpriv`	Private information about user authentication, normally accessible only to privileged users.
`cron`	Messages from the system scheduler `cron(8)`.
`daemon`	A catchall for processes that neither need nor require a dedicated facility.
`ftp`	Messages from FTP and Trivial File Transfer Protocol (TFTP) servers.
`kern`	Kernel-generated messages.
`local0` through `local7`	These facilities are provided for the sysadmin. Many programs let sysadmins configure their facility. Use these eight facilities for such programs.
`lpr`	Messages from the printing system.
`mail`	Messages from mail servers.
`mark`	This special facility writes a message every 20 minutes.
`news`	Messages from Usenet news servers.
`syslog`	Messages from the syslog server itself.
`user`	The catchall message facility. If a userland program doesn't specify a logging facility, the messages wind up here.
`uucp`	Messages from the Unix-to-Unix Copy Protocol (UUCP) servers. You will probably never encounter this pre-Internet email protocol.

While most programs have sensible defaults, it's your job as the system administrator to manage which programs log to which facilities. If possible, use the local facilities for your server-specific daemons. While it's entirely

possible to use facilities for purposes other than originally intended, try not to reassign the `uucp` facility to some other daemon unless you really have no other option.

Priority

A log message's priority represents its importance. Programs usually send their logging data to `syslogd`, but `syslogd` decides what to retain and what to discard. You get to decide how much detail you want in your logs. Use the following nine `syslog` levels to decide what to record and what to discard (in order from most to least important):

`emerg` System emergency. This message appears on every active terminal. The computer might be crashing, or it may have some other error that requires immediate attention.

`alert` An emergency. The system can continue to function, but attend to this error very soon.

`critical` Critical problems. These indicate serious errors, such as hard-drive failures.

`err` Errors. These are in regard to problems that require attention but won't destroy your system.

`warning` Miscellaneous warnings. These could be attended to, but will not prevent the process that generated them from running normally.

`notice` Important information, such as daemon startup and shutdown notifications.

`info` Basic information. This usually includes transactional data, such as individual messages in a mail server or individual queries to a web server.

`debug` Trivia. This level is usually of interest only to programmers, but occasionally useful to sysadmins trying to figure out why a program is behaving in a certain way. Debugging logs can contain anything, including information that violates user privacy, such as plaintext passwords.

`none` Don't log anything from this facility here. This is most commonly used to exclude information from log files, as discussed shortly.

By combining levels with priority, you can sort log messages into individual files or other targets.

Sorting Messages via syslogd(8)

`syslogd` compares received messages to entries in */etc/syslog.conf*. This file has two columns: the first (the *selector*) describes a type of log message, and the second (the *action*) tells `syslogd` what to do when a message matches the description. Neither column can have whitespace; whitespace can appear only between the columns. For example, here's a line from the default *syslog.conf*:

```
daemon.info             /var/log/daemon
```

Any log message that has a facility of daemon and a priority of info or higher is appended to the file */var/log/daemon*. Of course, if all logs were so easily managed, this would be a short section.

syslogd compares all log messages to all *syslog.conf* entries. If a log message matches multiple selectors, it is sent to all matching destinations.

Wildcards

You can use wildcards in either the facility or priority. For example, this line logs every message from the mail facility:

```
mail.*              /var/log/maillog
```

To capture messages of a given priority or higher from all facilities, use an asterisk (*). Here's how to send all priority err and higher messages to the console:

```
*.err    /dev/console
```

You can also use a double-wildcard to send all log messages to one place.

```
*.*    /var/log/all.log
```

Logging everything to one location isn't terribly useful or wise. You should *not* send authpriv debugging to a world-readable file.

Excluding Information

Use the none level to exclude information from a log. For example, the following line excludes private authentication information from an otherwise all-inclusive log.

```
*.*;authpriv.none    /var/log/most.log
```

The semicolon (;) allows you to combine selection criteria on a single line.

WARNING *If you combine entries with a semicolon like this, do not put whitespace after the semicolon. The only whitespace can appear between the selector and the destination.*

Combining Facilities

You can combine multiple facilities in a single entry by using commas. Here's how to capture all messages of info priority or higher from several facilities:

```
auth,daemon,syslog,user.info    @loghost
```

Any log message from the auth, daemon, syslog, and user facilities, and of priority info or higher, is sent across the network to the host loghost.

Marking Time

While all log messages have a timestamp, you might want a marker in a log file to indicate that time has passed. The special facility `mark` creates a message every 20 minutes, letting you add an extra timestamp to a file. Here's how to add a timestamp to the mail log every 20 minutes:

```
mail.info;mark.info    /var/log/maillog
```

Local Facilities

The eight facilities `local0` through `local7` are for your use. Many programs can be configured to use a specific facility, so you can aim them at a particular file. I've configured a daemon to use the facility `local7`. Here, I send messages from that facility to a file:

```
local7.*    /var/log/postgres.log
```

Some programs have a hard-coded preference for a specific facility. For example, the flow-tools package (see my book *Network Flow Analysis*, No Starch Press, 2010) has facility `local6` hard-wired into the code. Don't be shocked when you see something like this. Fortunately, OpenBSD's `syslogd` can filter based on program name, so you can easily filter your logs despite this sort of daftness.

Selecting by Program Name

If you're out of facilities, you can use the name of the program generating the syslog messages as a selector. Using a program name requires two lines: The first contains the program name with a leading exclamation mark, and the second sets up logging. OpenBSD offers the following example of `sudo(8)` logging:

```
!sudo
*.*    /var/log/sudo
```

All log messages from `sudo` go to the specified log file.

You can also select by program name and stop all subsequent selections of matching messages by using two exclamation points (`!!`) before the program name. This example sends all messages from `sudo` to */var/log/sudo*, but prevents `sudo` messages from going to any other log.

```
!!sudo
   *.*    /var/log/sudo
!*
...
```

The !* after the end of the sudo entry is a way to say "all programs"—in other words, don't sort by program name anymore. You need this only if you use the double-exclamation-point "stop processing matching messages here" syntax.

Log Actions

Now that you know how to sort your log messages into different buckets, let's see how to take different actions with those messages. Messages can be written to a file, piped to a program, sent to another host, or written to users.

Logging to Files

Most of our examples so far send log messages to a file, giving the full path to the file as the action. Here's how to send all of the messages from facility local6 to a log file:

```
local6.*    /var/log/flowtools
```

You can also send the messages to a device by giving the full path to the device node, but this will make sense to very few devices, such as the console. This is because writing the log message to the disk device */dev/wd0d* will not store the message on disk.

Logging to a Program

To send selected logs to a program, use a pipe (|) and the full path to the program, like this:

```
*.*    |/usr/local/bin/logsurfer
```

The logging system should start the destination program, and then feed log messages into the program's standard input.

Notifying Users

You can also direct log messages to logged-in users by listing multiple users in a comma-separated list. For example, to send a message to all users, use an asterisk.

```
*.emerg    *
*.info    lasnyder
```

This example will notify all logged-in users of real emergencies, but deeply annoy lasnyder.[1]

1. Hey, I was running out of ways to annoy lasnyder—plausible ways, at least.

Logging to a Remote Host

I usually have a logging host that collects log messages from everywhere—not only from my OpenBSD boxes, but from all my other Unix-like systems, as well as routers, switches, and anything else that speaks syslog. This reduces my maintenance needs and conserves disk space. And, since, every log message includes a hostname, I can easily sort them out later.

To send messages to another host, use the `@` symbol.

```
*.info    @loghost.blackhelicopters.org
```

This dumps everything of priority `info` and above to my logging host.

Your logging host must accept syslog messages from the network. If your host is an OpenBSD machine, run `syslogd` with the `-u` flag. And be sure to protect your log host with a packet filter, so random hosts can't write logs to it and fill up your disks.

Customizing syslogd

OpenBSD runs `syslogd` by default, and you can customize how `syslogd` behaves. Common customizations include adding more log sockets and listening to the network.

Adding Extra Log Sockets

Programs write log messages to the socket */dev/log*, but software inside a `chroot` won't be able to access that device. To have a program that's locked inside a `chroot` send messages to `syslogd`, you must put an additional log socket at */dev/log* inside the `chroot`.

For example, since the integrated BIND DNS server is `chroot`ed into */var/named*, the DNS server expects to find the log socket at */dev/log*, which means that the new log socket should be at */var/named/dev/log*. To create this log socket, use `syslogd`'s `-a` option, and give the full path to the log socket in */etc/rc.conf.local*.

```
syslogd_flags="-a /var/named/dev/log"
```

You can use about 20 additional logging sockets.

Listening to the Network

If you want your OpenBSD box to act as a log host, accepting logs from remote hosts, use the `-u` flag.

```
syslogd_flags="-u"
```

Because the syslog protocol has no access control, anyone with access to port 514/UDP on the log host can write to your log files.

NOTE *Filling a host's logs with junk to fill the hard disk is an old attack. Use OpenBSD's packet filtering system (discussed in Chapters 21 and 22) to protect your logging host.*

Syslog and Embedded Systems

OpenBSD supports writing log messages to an in-memory buffer, which allows logging on systems that have no writable disk, such as diskless systems and embedded routers and firewalls. `syslogd` retains these logs in a memory buffer, and clients can connect to `syslogd` through a reporting socket and read the logs. As you would expect, logs in memory disappear when `syslogd` is shut down.

To use `syslogd` for reporting, first provide a reporting socket with the `-s` option and give it a full path to a reporting socket. Here's an *rc.conf.local* entry for a reporting socket in */var/run/syslog*:

```
syslogd_flags="-s /var/run/syslog"
```

To log to the buffer, make a *syslog.conf* action. Specify logging to a buffer with a colon (`:`), the number of kilobytes to give the buffer, another colon, and the name of the memory buffer. (The maximum buffer size is 256KB.)

For example, here we capture all log messages of `err` priority or higher and write them to the 128KB memory buffer called `errors`:

```
*.err    :128:errors
```

Use `syslogc(8)` to read a memory buffer, and use the `-s` option to tell `syslogc` where to find `syslogd`'s reporting socket, and provide the name of the log buffer. Here's how to read the reporting socket */var/run/syslog* and read the `errors` buffer:

```
$ syslogc -s /var/run/syslog errors
```

If you've forgotten the name of the buffer you want to read, ask `syslogc` to query the list of available memory logs with `-q`. Be sure to provide the reporting socket.

NOTE *Even if you're not a programmer, you can still use real syslog features. Logging to syslog is available to shell scripts via the `logger(1)` program. See the `logger` man page for details.*

Log File Maintenance

You can capture logs. Fantastic! Now just let the log files grow until they fill your hard disk and leave room for nothing else, right? Or you can discard old logs and have the system keep the logs to a manageable size. This is called *log rotation*.

Look at the system messages log, */var/log/messages*, and you should see six *messages* files: *messages*, *messages.0.gz*, *messages.1.gz*, *messages.2.gz*, *messages.3.gz*, and *messages.4.gz*. The plain *messages* file is the current log file. The other files are older logs; *messages.0.gz* is the newest, and *messages.4.gz* is the oldest.

When the current log file hits either a certain age or a specific size, log rotation discards the oldest log file (*messages.4.gz*), and the second-oldest file, *messages.3.gz*, is renamed to *messages.4.gz*; *messages.2.gz* is renamed to *messages.3.gz*; and so on. The existing *messages* file is renamed to *messages.0* and compressed, and a new *messages* file is created.

The `newsyslog(8)` program rotates log files, restarts daemons, runs commands, shuffles old files into other directories, and handles all routine tasks. `root` runs `newsyslog` once per hour via `cron(8)`. When `newsyslog` starts, it reads */etc/newsyslog.conf* and examines each log file listed. If the conditions for rotating the log file are met, the log is rotated and other configured actions are taken.

newsyslog.conf Fields

newsyslog.conf uses one line per log file. Each line has seven fields, like this:

```
/var/log/authlog        root:wheel      640  7     *    168   Z
```

From left to right, the fields are log file, owner, permissions, number of files to retain, size, time, and flags. After the flags field, you might see a number of optional arguments. We'll look at each of the fields in order.

Log File

The first entry on each line is the full path to the log file to be processed (`/var/log/authlog` in this example). This must exactly match the current log file.

Owner

The second field (`root:wheel` in our example) lists the log file's owner and group, separated by a colon. This field is optional, and is not present in many of the default entries.

By default, log files are owned by the root user and the `wheel` group, but `newsyslog` can change the owner of log files. While changing ownership isn't common, you might want to explicitly declare it for specific files.

You can choose to change only the owner or only the group. In these cases, use a colon with a name on only one side of it, such as `:wheel` or `root:`. You must always include the colon if you're changing ownership.

Permissions

The third field (`640` in our example) gives the rotated file's permissions in standard octal notation, as discussed in `chmod(1)`. This field is optional, and it is not present in many default entries.

Count

The fourth field specifies the number of archived log files that *newsyslog* keeps. In our example, */var/log/messages* has the current log file and five archives, numbered 0 through 4. *newsyslog.conf* has a count of `5` for */var/log/messages*.

Size

The fifth field is a file size in kilobytes. When `newsyslog` runs, it checks the size of the log file. If the log is larger than the size given here, `newsyslog` rotates the log. If you don't want the file size to affect when `newsyslog` rotates the file, put an asterisk here.

Time

To rotate the log based on time, use the sixth field, which has four possible values: an asterisk, a number, and a time in one of two standard formats. If you rotate the log based on size rather than age, put an asterisk here. If you put a number here, you are specifying a number of hours after which the log will rotate. Our example of */var/log/authlog* rotates every 168 hours.

The time formats—ISO 8601 restricted and `newsyslog`-specific—are a little more complicated.

ISO 8601 restricted

A time entry beginning with an `@` symbol is in the ISO 8601 restricted time format. The ISO 8601 restricted time format is used by `newsyslog` on most Unix-like systems, because it was the time format used in MIT's primordial `newsyslog`. The ISO 8601 format is a bit obtuse, but every Unix-like operating system I'm aware of supports it.

A full date in ISO 8601 format is 14 digits with a `T` in the middle. The first four digits are the year, the next two are the month, and the next two are the day of the month. (The `T` serves as a sort of decimal point, separating whole days from fractions of a day.) The next two digits are hours, the next two are minutes, and the last two are seconds. For example, the date September 13, 2013, at 3:18 and 58 seconds PM is expressed as `20130913T151858`. (Specifying a specific date and time to rotate a log wouldn't be terribly useful because the log would rotate only once.)

You can choose to specify only the fields near the `T`, leaving fields farther away blank. Again, if you think of the `T` as a decimal point, you don't need to write 5.87 as 005.8700; the leading and trailing zeros are irrelevant.

In the case of `newsyslog`, empty fields are wildcards. For example, `4T00` matches midnight on the fourth day of every month, and `T23` matches the twenty-third hour, or 11 PM, every day. If *newsyslog.conf* lists the time `@T2359`, the log rotates at 11:59 PM every day. (Of course, `newsyslog` runs once an hour, so the log won't rotate exactly then.)

As with cron(8), specify time units in detail. For example, @9T, the ninth day of the month, rotates the log once an hour, every hour, on the ninth day of the month, which would mean it rotates the log all day on that day. It would probably be better to specify a time of @9T01, which would rotate the log at 1 AM on the ninth day of the month. You don't need to specify times any more closely than the hour, as newsyslog runs only hourly.

newsyslog times

Because ISO 8601 time doesn't let you easily specify weekly jobs, and it's impossible to specify the last day of the month, OpenBSD includes a newsyslog-specific time format that lets you easily specify these common times.

Any entry with a leading dollar sign ($) is written in *month week day* format.

This particular format uses three identifiers: *M* (day of month), *W* (day of week), and *H* (hour of day). Each identifier is followed by a number indicating the unit you're using. Hours range from 0 to 23, and days run from 0 (Sunday) to 6 (Saturday). Days of the month start at 1 and go to 31, with L or l representing the last day of the month. For example, to rotate a log on the fifth of each month at noon, use $M5H12. To start the month-end accounting at 11 PM on the last day of the month, use $MLH23.

If you don't specify an hour, the time defaults to midnight on the chosen day. And if a *newsyslog.conf* entry lists both a time and a size for file rotation, newsyslog rotates the log if either requirement is met.

Flags

The seventh field, which is optional, instructs newsyslog on special processing for the file itself. OpenBSD uses four flags:

Z Compress the file with gzip(1).

B Do not add a "log file turned over" message to the file (for binary files).

F Follow symlinks.

M A user is monitoring this log.

While the B and Z flags are not, strictly speaking, mutually incompatible, most log files need only one of them, and most binary files don't compress well anyway. (The default *newsyslog.conf* compresses the packet filtering log file, but that's something of an oddity.) If you see the Z flag with the M flag, the old log file will be sent to the user before the log is compressed.

Monitoring Logs

OpenBSD's `newsyslog` can email logs to a user before rotating them. If you carefully control how you sort your logs, this feature can be useful. For example, `sudo(8)` logs successful uses at priority `notice`, but failed uses at priority `alert`. You might split these into separate log files in *syslog.conf*, like this:

```
!sudo
*.*         /var/log/sudo
*.alert    /var/log/sudofail
```

The file */var/log/sudofail* should now contain only `sudo` failures, such as users entering incorrect passwords or exceeding their privileges.

Now you could tell `newsyslog` to check for monitored logs by running it with the `-m` flag. (`newsyslog` runs as one of root's cron jobs.)

To have the `sudo` failure log emailed to you every time the log rotates, you can put your account in the monitor field.

```
/var/log/sudofail    root:wheel    640    30    *    $H06    ZM    mwlucas
```

This assumes that email to the account `mwlucas` on this machine reaches me. The simplest way to ensure that would be to forward the email in */etc/mail/aliases*.

NOTE *If you're serious about watching these kinds of failures, monitor logs on a logging host that end users cannot access. A user who becomes root on the local machine can edit logs before they are emailed and rotated.*

Adding a PID File

If `newsyslog` tries to rotate and compress a file, but the process writing the file is still writing to the file, the file can become corrupted. Some programs need a right proper slapping before they will let go of their log files. How? Just list a PID file here, and `newsyslog` will send that process ID a `SIGHUP` (like a `kill -1`).

Note that PID files are not a terribly secure way to identify specific processes because they are subject to race conditions and other attacks. If the server has a command for rotating its logs, that's probably a wiser choice than signaling a process indicated in a PID file.

Signal Name

To send a signal other than `SIGHUP` to a process with a PID file, use a different *signal name*. The signal name must begin with `SIG` and be specified by name. You can find a full list of signals in `signal(3)`, but the software

documentation should tell you which signal the process needs to release in order to restart its log file. This field is optional, but if you use it, you must enter a full path to a PID file immediately before it.

Command to Execute

Rather than signaling a process, you can have `newsyslog` run a command when rotating logs by giving the full path to the command in double quotes. While this field is optional, it cannot be combined with a PID file. You can use a PID file or a command name, but not both.

System Time

There's no excuse for a system having incorrect time. Once you set the time zone, having OpenBSD correct its own clock on an ongoing basis from any number of freely available network time servers is easy. Virtual machines in particular are notorious for skewing clocks, but time correction works on them as well, so as I said, no excuse.

OpenBSD includes its own NTP client, OpenNTPD, which is written to be safe and secure. Before `ntpd(8)` can do anything though, it needs some configuration.

Configuring ntpd(8)

OpenBSD comes with a perfectly acceptable generic `ntpd` configuration that uses public time servers. If your host is on the public Internet and you only want to set your system time, not provide time to other hosts, use the defaults. Otherwise, you must customize */etc/ntpd.conf* by selecting time sources and deciding if `ntpd` will accept time requests from other machines.

Time Redundancy

NTP gets the time by querying remote servers. If you have a single server, that time is assumed to be correct. However, if you have multiple time servers, the times are not simply averaged. If one time server is wildly off from all of the other time servers, the results from that server are discarded, and a median from the remainder is selected. If you have only two time servers, and the times obtained from them differ, `ntpd` can't determine which one is correct. To help `ntpd` make sensible decisions, always list at least three time servers.

Time Sources

Choose your time sources with the `server`, `servers`, and `sensor` keywords.

The `server` option tells `ntpd` to get the time from a single server, which might have multiple IP addresses. If that's the case, `ntpd` tries to use the first

IP address. If the first address doesn't work, it tries the second, and so on, until it gets an answer. Use the `server` option if you have specific time servers to use, and be sure to list at least three time servers.

```
server time1.blackhelicopters.org
server time2.blackhelicopters.org
server time3.blackhelicopters.org
```

The `servers` option tells `ntpd` to get the time from multiple hosts that share a common hostname. The default *ntpd.conf* includes this entry:

```
servers pool.ntp.org
```

The host *pool.ntp.org* has four IP addresses, and `ntpd` will try to get time from all of those hosts.

If you have a hardware time sensor, you can tell `ntpd` to read time from it. Hardware time sensors include `nmea(4)`, `udcf(4)`, and `mbg(4)`. The `sensor` option tells `ntpd` to use a hardware sensor. If you've invested in a hardware time sensor, you might be sufficiently concerned about time to measure the distance between the transmitter and the receiver, and adjust the time based on the speed of light delay. The `correction` keyword lets you specify a number of microseconds that your sensor is behind.

As I wrote this, a 40-millisecond (ms) delay caused a furor in the scientific world when researchers thought they might have seen a neutrino go faster than light, so we'll put a 40 ms correction into our time sensor.

```
sensor nmea0 correction 40000
```

NOTE *Be warned that OpenBSD is not a real-time operating system. You should not be measuring neutrino speed with it anyway!*

Serving Time

I run time servers on only closed networks, where very few hosts have access to the public Internet. I would also run time servers if I had hardware time sensors, but most of the time, I just use the public time servers.

To have `ntpd` answer time queries from other hosts, use the `listen on` directive. You can either specify an IP address or use an asterisk to say "every IP on the system."

```
listen on 192.0.2.87
listen on 2001:db8::aaaa
```

Because `ntpd` has no access controls, any host that can connect to port 123/UDP can get time from this server. If this worries you, use packet filtering (discussed in Chapters 21 and 22) to limit time checks to hosts on your

network. The author of OpenNTP served time to his entire company and to the public on a MicroVAX 3100 with 16MB (yes, that's an *M*) of RAM without the NTP process using more than 5 percent of the processor, so the load imposed by NTP is negligible on modern systems.

Now that `ntpd` is configured, let's use it.

Using ntpd(8)

You can correct time slowly or do it in one fell swoop. I recommend fully correcting time at boot, and then letting `ntpd` slowly adjust the system clock as the system runs. This corrects time before anything relies on it, but keeps everything synchronized on an ongoing basis.

To correct time when starting `ntpd`, use the `-s` flag.

```
# ntpd -s
```

You'll get a command prompt back once `ntpd` receives a response from a time server and adjusts the clock. At boot, this delays other software starting so that it has the correct time, and when you check your clock, you should see the correct time. You can configure this at boot with `ntpd_flags` in */etc/rc.conf.local*.

```
ntpd_flags='-s'
```

If the clock is off on a running system and you're running software that would be corrupted by the clock moving backward or a time jump forward (as with many databases), you might need to tell `ntpd` to correct the clock more slowly. To do so, run `ntpd` without any flags, or set it in *rc.conf.local* to have it run in this mode at boot.

NOTE *Your starting time may be so far off that it will be impossible to make a gradual adjustment to the correct time in any reasonable period. To fix the clock, schedule a clock change when you can shut down your sensitive software, and make sure NTP runs afterwards so that the problem remains fixed. It's better to fix your clock right away and be done with it.*

Hardware Sensors

Sensors are physical probes that check the health and status of hardware. Manufacturers have put more and more sensors in hardware, providing low-level hardware information to the operating systems. OpenBSD supports a wide variety of hardware sensors, and uses the `sensorsd` daemon to query them and act upon error states.

Resolving many hardware errors requires shutting down the machine, but advance warning that a component has stopped working changes a hardware failure from an unexpected middle-of-the-day catastrophe to an

after-hours annoyance. Some hardware, such as hot-swappable hard drives, can be replaced without interrupting service once you know the hardware has failed.

Device Drivers

Each physical sensor has a device driver. The device driver extracts information from the hardware and publishes it in a sysctl (discussed in Chapter 18). `sensorsd` reads the sysctl values and can act when they change or cross critical values. For example, here are the sensor-related sysctl values from my laptop:

```
$ sysctl hw.sensors
hw.sensors.acpitz0.temp0=67.00 degC (zone temperature)
hw.sensors.acpiac0.indicator0=On (power supply)
hw.sensors.acpibat0.volt0=11.10 VDC (voltage)
hw.sensors.acpibat0.volt1=12.35 VDC (current voltage)
hw.sensors.acpibat0.power0=0.00 W (rate)
hw.sensors.acpibat0.watthour0=2.61 Wh (last full capacity)
hw.sensors.acpibat0.watthour1=0.30 Wh (warning capacity)
hw.sensors.acpibat0.watthour2=0.06 Wh (low capacity)
hw.sensors.acpibat0.watthour3=9.57 Wh (remaining capacity), OK
hw.sensors.acpibat0.raw0=2 (battery full), OK
hw.sensors.cpu0.temp0=81.00 degC
```

This comparatively simple and generic hardware has two temperature sensors and all kinds of power sensors. You can get hundreds of lines of sensor output, depending on your hardware.

Many RAID controllers have their own sensors, and will report when an array has failed. Here, we see three virtual disks provided by an AMI RAID controller:

```
hw.sensors.ami0.drive0=online (sd0), OK
hw.sensors.ami0.drive1=degraded (sd1), WARNING
hw.sensors.ami0.drive2=failed (sd2), CRITICAL
```

If you didn't have sensors, you would need to look at the blinking lights on the drive enclosure. Or you could listen for the really annoying "beep, beep, beep," which is *so* easy to hear over the roar of 5,000 server fans, the air conditioners, and someone else's hardware that has been beeping every time you've come in for the last six months.

NOTE *Some sensors require the Intelligent Platform Management Interface (IPMI). This is a kernel feature that's disabled by default in OpenBSD, because it makes some machines behave really badly. Chapter 18 discusses enabling IPMI.*

The device drivers attach to sensors automatically, and the values get into the kernel automatically, but to do anything with these results in any

automated manner, you need `sensorsd(8)`, or you need to configure an external SNMP-based management system and use `snmpd(8)`. We'll look at using `sensorsd(8)` here. Using `snmpd(8)` is discussed in Chapter 16.

Sensor Configuration

The sensors daemon `sensorsd(8)` watches sensor monitoring data. It logs changes and can execute commands if needed. Because all hardware is different and all environments are different, by default, `sensorsd` notices changes only in sensor readings. To take action, you must configure `sensorsd` in */etc/sensorsd.conf*.

Sensor Types

OpenBSD supports many types of sensors, as listed in Table 15-2.

Table 15-2: Supported Sensor Types

Name	Function
temp	Temperature (C)
fan	Fan speed (RPM)
volt	DC voltage
acvolt	AC voltage
resistance	Ohms resistance
power	Wattage
current	Amperage
watthour	Power capacity
amphour	Power capacity
indicator	Device-dependent yes/no
raw	Device-dependent value
percentage	Device-dependent percentage
illuminance	Lighting
drive	Hard drives
timedelta	Time difference between operating system and hardware
humidity	Percent humidity
frequency	Microhertz
angle	Microdegrees

You'll need to check your hardware manual in order to learn how to use some of these sensors effectively.

Some sensors appear to overlap. For example, why does OpenBSD have all those separate values for power, when you could probably do some math and get a common power gauge? The reason is that these are the values that the actual sensors report, and the developers would prefer to give you

the actual measurements. OpenBSD does perform some data rationalization, but only for simple data; all temperature sensors are normalized to degrees Celsius, for example.

Now let's see what you can do with these sensors.

Settings in sensorsd.conf

The file *sensorsd.conf* has example entries, but because environments differ so widely, they're all commented out. It uses a `termcap`-style configuration syntax, much like */etc/remote* (see Chapter 5) or */etc/login.access* (see Chapter 6), with colons separating the terms in an entry. Each entry starts with the sensor to be measured, followed by attribute names and settings.

For example, here's an entry for a temperature sensor in the default *sensorsd.conf*:

```
hw.sensors.lm0.temp0:high=50C
```

For the sensor `lm0.temp0`, the attribute `high` is set to `50C`.

`sensorsd` supports four attributes:

`high` An upper limit

`low` A lower limit

`command` A command to run when a limit is crossed or a state changes

`istatus` Ignore this status

The values reported for a sensor type depend on what makes sense. Where high and low limits make sense for temperature and voltage, some sensors report specific values instead. The RAID controller shown earlier reports drives as degraded, failed, or healthy. A hard-drive sensor that reports a scalar value isn't useful, as you want to know if a RAID container is healthy or if drives have failed. There's no middle ground.

You can have both high and low values for a single sensor. For example, whereas temperature might not have a low value in most data centers, voltage certainly will. I work in all sorts of weird places, and not all of them have clean power.

```
hw.sensors.acpibat0.volt0:low=11.0V:high=13.0V
```

With a line like this, if the electricity supply to my laptop drops below 11 volts or goes above 13 volts, I will know.

Some systems might have dozens of sensors of a given type, which could make configuration tricky. If my motherboard has 15 temperature sensors, I don't want to configure each separately. Fortunately, you can configure sensors en masse by type, and since I don't care which temperature sensor goes above 80 degrees Celsius (if any of them do, I want an alarm), that works.

```
temp:high=80C
```

When this rule is applied, `sensorsd` first looks for a configuration item for a specific sensor. If it doesn't find that specific rule, it looks for a general rule. You can have one rule for most of your temperature sensors, and then override it for specific sensors, like this:

```
hw.sensors.lm0.temp5:high=90C
temp=80C
```

This rule says that most of my temperature sensors alarm at 80 degrees, but one specific sensor doesn't alarm until 90 degrees.

I care about temperature, but I don't care if my fancy keyboard sees that there's no light and wants to trigger its back lighting. You can ignore a sensor, or a type of sensor, with the `istatus` keyword.

```
illuminance:istatus
```

You should categorically ignore certain types of alarms based on your environment and gear. Make up your own mind.

Sensors Triggering Action

Having an entry in */var/log/daemon* for when a hard drive fails is nice, but it would be better if the system would send email, page you, or trigger your monitoring system. It should do something—*anything*—that doesn't require you to log in and look at a log file. Fortunately, `sensorsd` can run arbitrary commands upon detecting a problem or crossing a threshold, using the `command` attribute.

Thanks to the wide variety of sensors and their possible error states and conditions, `sensorsd` doesn't have a fine-grained "run this command for an error, but run that other command for recovery." There are too many possible error states and conditions for this to make any sense. Instead, `sensorsd` runs a single command upon crossing any threshold or upon any state change, including when it starts up and the state of an individual sensor goes from "unknown" to whatever it starts at.

Consider this *sensorsd.conf* entry:

```
temp:high=80C:command=/sbin/reboot
```

At first glance, this reads "If the temperature is high, reboot the machine." You think that will unquestionably kill whatever runaway process is saturating your heat-generating CPU (completely setting aside the fact that other hardware besides CPUs generate heat), but `sensorsd` will run the command whenever the temperature state changes. The state changes at boot time, when the first temperature reading is taken, which means that your system will boot, and then immediately reboot. Your script needs intelligence.

To make scripting easier, `sensorsd` has a set of variables it can pass to a script:

`%1` Is the value within the limit set in *sensorsd.conf*? This can be one of `below`, `above`, `within`, `invalid`, or `uninitialized`.

`%n` Sensor number.

`%s` Sensor status.

`%x` Which device the sensor sits on.

`%t` Sensor type.

`%2` Sensor's current value.

`%3` Sensor's low limit

`%4` Sensor's high limit.

You might run a temperature command like this:

```
temp:high=80C:command=/usr/local/script/temp %1 %2 %n
```

Your script */usr/local/script/temp* would take three arguments: the error condition, the temperature, and the sensor name. Your script would check these values and see if a reboot is warranted.

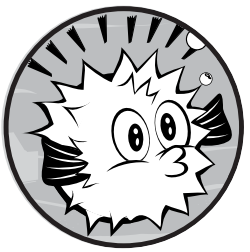
With `sensorsd`, proper timekeeping, and log file management, your OpenBSD system can largely look after itself.

In the next chapter, we'll look at how OpenBSD can take care of other hosts.

16

NETWORK SERVERS

Working behind scenes,
taking care of vital things,
the daemon is here.



The OpenBSD base system includes several servers to support a network. This chapter covers the following network servers:

- Small-server handler `inetd`
- Printer daemon `lpd`
- DHCP daemon `dhcpd`
- TFTP daemon `tftpd`
- SNMP agent `snmpd`
- SSH server `sshd`

This miscellany of small daemons supports the features covered in upcoming chapters.

The inetd Small-Server Handler

The inetd(8) "super-server" handles incoming network requests for network services that aren't used very often. After all, many systems don't have a steady stream of incoming FTP requests, so why have an FTP daemon running constantly? Instead, inetd listens for incoming network requests, and when an FTP request arrives, it starts the FTP server and feeds it the request. Other common services that frequently (but not always) run through inetd include ident, finger, and TFTP. Many of these services can also run standalone, if the application usage warrants it.

inetd also handles functions so small and rarely used that they're easier to implement within inetd itself, rather than by calling a separate program. These functions include discard (which dumps any data received into the bottomless pit of */dev/null*), chargen (which pours out a stream of characters), and echo (which repeats whatever you send to it). Most of these services are not needed on the modern Internet and are disabled by default, but you have access to them if necessary.

Configuring inetd

You configure inetd in */etc/inetd.conf*. Here's the default inetd configuration for OpenBSD's FTP server:

```
#❶ftp ❷stream ❸tcp ❹nowait ❺root ❻/usr/libexec/ftpd ❼ftpd -US
#ftp    stream   tcp6  nowait   root   /usr/libexec/ftpd   ftpd -US
```

The first thing you'll notice is that these entries are commented out. OpenBSD's default inetd offers only the identity server identd(8) and two time services by default.

The first field is the service name (ftp in this case) ❶. The name in this field must match a name in */etc/services*. The inetd program uses the *services* file to perform a service lookup to identify which ports it must listen on. To change the TCP/IP port that your FTP server runs on, change the port for FTP in */etc/services*. (You could also change the first field to use the name of the service that usually runs on the desired port, but I find starting my FTP server entry with the wrong name just gives me a headache.)

The second field is the socket type (stream in this case) ❷. This field dictates what sort of connection this is. All TCP connections are of type stream, and UDP connections are of type dgram. The inetd program does support other types of connections, but they're rarely used. If you're considering using them, either you're reading the documentation for a piece of software that needs that type of connection or you're wrong (probably the latter).

The third field is the layer 4 network protocol, usually tcp ❸, udp, tcp6, or udp6. If you want to offer a service over both IPv4 and IPv6, you need a separate entry for each. That's why there are two otherwise identical configurations for the FTP server. The inetd program also supports RPC services, which have type rpc/udp or rpc/tcp.

The fourth field (`nowait` in this case) ❹ indicates whether `inetd` should wait for the server program to close the connection or just start the program and go away. As a general rule, TCP-based daemons use `nowait`, and UDP-based daemons use `wait`. (There are rare exceptions.)

The fifth field (`root` in this case) ❺ names the user that the server daemon runs as. Many `inetd`-using programs must run as root, as they can affect multiple users or accept more specific logins, but some smaller programs have dedicated unprivileged users.

The sixth field is the full path to the server program `inetd` runs when a connection request arrives ❻. Services implemented within `inetd` have a path of `internal`. The FTP server is at */usr/libexec/ftpd*.

Finally, the last field gives the command to start the server program, including any command-line arguments you want. This configuration runs the FTP server with the arguments `-US` ❼.

Restricting Incoming Connections

Script kiddies occasionally try to knock a server off the Internet by sending it more connection requests than it can handle. The `inetd` program accepts up to 256 connections per minute per service. If a service receives too many connection requests, `inetd` logs the issue and stops answering requests for that service for 10 minutes.

NOTE *The IPv4 and IPv6 versions are limited separately, so you could accept 512 FTP connections per second if the requests are evenly divided between protocol families. You can override this globally with a command-line flag when starting* `inetd`*, or you can configure this on a per-service basis.*

The `-R` flag controls how many connections per minute and per service that `inetd` accepts. For example, to accept 1000 requests per minute, you would set the following in */etc/rc.conf.local*:

```
inetd_flags='-R 1000'
```

You can set per-service limits by editing the `wait/nowait` field in the service's *inetd.conf* entry. Add a dot to the `wait` or `nowait` entry, followed by the number of times per minute you want to allow the service to be called. For example, if you have an FTP server that should be used by only a few of your friends, you could limit the server to 10 requests per minute, as follows:

```
ftp   stream  tcp    nowait.10  root    /usr/libexec/ftpd       ftpd -US
ftp   stream  tcp6   nowait.10  root    /usr/libexec/ftpd       ftpd -US
```

Now, if more than 10 connection requests arrive in one minute, `inetd` stops servicing FTP requests for ten minutes. An attacker could still use this to knock your FTP service offline, but not to knock the entire server offline. At least this way you get to choose your failure mode and when you reach it.

The lpd Printing Daemon

OpenBSD includes the `lpd(8)` printing daemon common on Unix-like operating systems. The `lpd` daemon has options to support thousands of different printers, but getting the right mix of options to support any one specific printer can be a challenge.

The simplest way to use a printer on OpenBSD is through a PostScript server, and that's the method I'll cover here. Many modern printers, particularly the popular multifunction fax/scanner/printer combinations, support PostScript, and you'll find that every office print server does, too.

Every printer your system knows about needs an entry in */etc/printcap*, the printer capability database. This is another `termcap(5)`-style configuration file. You don't need to know everything about the printer to change settings here. This entry just needs the hostname or IP address of the print server and the print server's name for the printer you want to access. Then use the following template:

```
lp|printername:\
  :sh=:\
  :rm=printservername:\
  :sd=/var/spool/output/printername:\
  :lf=/var/log/lpd-errs:\
  :rp=printername:
```

The first line gives the printer's names. Every printer can have any number of names, separated by the pipe (`|`) symbol. The default printer on a Unix-like system is named `lp`, so be sure that one of the printers has that name attached to it. Another name should be the one used by the print server for this printer (such as `Billing`). (Microsoft print servers frequently share one printer under several different names, and each name prints differently, so be sure to use the name that represents the PostScript facility.)

The other lines list attributes:

- By default, `lpd` precedes each print job with a page giving the job name, number, host, and other identifying information. This used to be important when people paid for printing by the page, but unless you're in an environment with a single, massive printer, this probably wastes paper. The `:sh:=\` entry suppresses this page.
- The `:rm=` attribute gives the hostname or IP address of the print server. You must be able to ping the print server by this name.
- Printing works best if each printer has a unique spool directory, given by the `:sd=` attribute. The printer daemon stores documents en route to the print server here. This directory must be owned by the user root and the group daemon.
- Several printers can share a common log file, shown by the `:lf=` attribute.
- Finally, specify the remote printer name with the `:rp=` attribute. This last attribute is the only one that doesn't end with a backslash.

Always end */etc/printcap* with a newline. I usually use an entire blank line, just to be certain.

Now that you have a printer configuration, you start `lpd` at boot with this *rc.conf.local* entry:

```
lpd_flags=""
```

Restart `lpd` with `/etc/rc.d/lpd restart` any time you edit */etc/printcap*.

Finally, view the print queue with `lpq(1)`, and watch */var/log/lpd-errs* for problems.

The DHCP Server dhcpd

DHCP is the standard method for dynamically configuring clients on an IP network. You might know DHCP as a way to give computers basic IP information, but it can also hand out configuration files for embedded devices such as routers and phones, point diskless machines to their kernel and userland, and much more.

OpenBSD includes a heavily modified ISC DHCP server, `dhcpd(8)`. Here, we'll cover the basics of using `dhcpd` for configuring dynamic clients in a shared Ethernet system. In Chapter 23, we'll discuss the details of using DHCP to configure diskless workstations.

How DHCP Works

A client seeking DHCP information broadcasts a request across the local network asking for someone—*anyone*—to give it a network configuration. If your DHCP server is on that Ethernet segment, it answers directly. If it's on another network segment, the router for that network segment can forward the DHCP request to your server, which will then offer a configuration to the client, maintaining a list of which clients have been assigned which unique configuration values (such as IP addresses). A configuration issued to a client is called a *lease*. Like all leases, DHCP leases expire and must be renewed in order to be valid.

Clients can request certain DHCP features to support their operations. For example, Microsoft clients request the IP addresses of the network Windows Internet Name Service (WINS) servers, Voice over IP (VoIP) desktop phones request their configuration file, and diskless systems (discussed in Chapter 23) ask where to find their kernel and userland. The DHCP server can offer this information, or not.

The DHCP server uniquely identifies each client by the MAC address of the network card it uses to connect to the network. To find out what information a client received from the DHCP server, get the client's MAC address and search for it in the */var/db/dhcpd.lease* file.

Configuring dhcpd(8)

Configure `dhcpd` in */etc/dhcpd.conf*. The default *dhcpd.conf* file includes a sample configuration suitable for a small office environment, as well as a diskless client sample configuration.

I'm going to assume that you're running a single DHCP server on your network, and that this server is authoritative for DHCP services. (OpenBSD's DHCP server also supports clustering for fault tolerance.)

Before configuring `dhcpd` to configure clients dynamically, you'll need a few facts about your network:

- Domain name
- DNS servers
- IP network and netmask
- Range of IP addresses in the network used for DHCP clients
- Default router

Once you have this information, you can assemble a brief *dhcpd.conf*. Here's an example:

```
❶ option domain-name "blackhelicopters.org";
❷ option domain-name-servers 192.0.2.5 192.0.2.10;

❸ subnet 198.51.100.0 netmask 255.255.255.0 {
❹     option routers 198.51.100.1;
❺     range 198.51.100.51 198.51.100.100;
}
```

All hosts that get their configuration from this host are told that their domain name is *blackhelicopters.org* ❶, and that they should use the name servers 192.0.2.5 and 192.0.2.10 ❷. The client can be configured to ignore or override this DHCP configuration, but you can't prevent local sysadmins from hanging themselves.

Each subnet needs its own configuration. Even if you have only one subnet, you must still have a `subnet` statement defining the IP network for that subnet so that `dhcpd` can determine which clients get which configuration. This example defines the configuration for clients on the network at 198.51.100.0/24 ❸. Everything inside the brackets that follow applies only to hosts on this subnet.

The `routers` option at ❹ identifies the default gateway for this network. Because the `dhcpd` server won't let you define additional static routes to feed to clients, your local network router must have proper routes to reach the destination. If you have multiple gateways on your local network, your default router should send an ICMP redirect to the DHCP client to correct its routing. (You don't unilaterally block ICMP from your firewalls, do you?)

The `range` keyword gives the IP addresses that the DHCP server can assign to clients. In this example, the DHCP server controls the addresses 198.51.100.51 to 198.51.100.100, inclusive ❺. If 52 dynamic clients connect simultaneously, the last client won't get an address.

This configuration should get your clients on the network.

Static IP Address Assignments

You can tell your DHCP server to assign a specific address to specific hosts by specifying the Ethernet address of the client in the configuration and using a stanza within the `subnet` statement. Here's the earlier DHCP configuration with a static entry added:

```
subnet 198.51.100.0 netmask 255.255.255.0 {
    option routers 198.51.100.1;
    host lucas-desktop {
        hardware ethernet 00:cf:01:b1:9b:07;
        fixed-address 192.0.2.254;
    }
}
```

I've found the MAC address of my workstation, and used it to assign a static IP address to that machine. This client machine inherits the default router from the subnet definition, as well as any default DHCP information.

Enabling dhcpd

Enable `dhcpd` in *rc.conf.local*.

```
dhcpd_flags=""
```

If you have only one network-facing interface, `dhcpd` will automatically listen for DHCP requests on that interface. If you have multiple interfaces, give the interface name as an argument. For example, here's how to tell `dhcpd` to listen for requests only on the interface `fxp1`:

```
dhcpd_flags="fxp1"
```

The interface name must be the last `dhcpd` argument in *rc.conf.local*. If `dhcpd` needs to handle several interfaces, the list of interfaces must come after any other arguments in `dhcpd_flags`.

dhcpd and Firewalls

The OpenBSD packet filtering system includes *tables*, which are lists of IP addresses that the packet filter applies rules to. Traffic from IP addresses in tables can be blocked, have its bandwidth throttled or prioritized, or be allowed to pass. Each table has a unique name.

The `dhcpd` server can add addresses to packet filter tables, thereby dynamically changing the firewall rules depending on whether an IP address is leased. Here, we'll look at configuring `dhcpd` to give addresses to the packet filter tables. Chapter 21 discusses how to configure the packet filter to handle addresses from `dhcpd`.

DHCP considers IP addresses in its address pool to be in one of three states: leased, abandoned, or changed. *Leased* addresses are addresses

assigned to a host attached to the network. Use `-L` to give `dhcpd` the name of the packet filter table for leased addresses, and then configure the packet filter to allow or deny those addresses access to the rest of the network.

Abandoned addresses are ones that have been assigned to a host, but that are not currently in use. In practice, that means that if you shut down your laptop, the DHCP server will consider the IP address assigned to it abandoned. The problem with that is that unauthorized users might try to get on the network by taking an unused address from the address pool, without going through the DHCP server. To address this problem, give the packet filter the list of addresses not in use, and give illicit network hosts their own special packet filter rules. Use the `-A` argument to tell `dhcpd` the name of the packet filter table for abandoned addresses.

If a host changes its address despite the DHCP server's configuration instructions, the DHCP server considers the address *changed*, and `dhcpd` can add its new address to the changed address table. Use the `-C` argument to tell `dhcpd` the name of the changed address table. (In Chapter 21, we'll do something interesting with these tables.)

```
dhcpd_enable="-A table1 -L table2 -C table3 fxp1"
```

NOTE *Static IP address assignments do not go into tables. If you assign a static address to a host, you must manually configure firewall rules for that address.*

The TFTP Daemon tftpd

The Trivial File Transfer Protocol (TFTP) is used to transfer files across a network. Unlike FTP, TFTP doesn't include authentication. Anyone who can access the TFTP server can upload or download files from it.

TFTP is an inflexible protocol. It doesn't work through network address translation without a proxy or some kind of intelligence within the translation device, and there's no interactive session as there is with FTP and SFTP. TFTP is most commonly used to copy configuration files and operating system images for embedded devices such as routers.

OpenBSD uses TFTP to bootstrap diskless systems, as discussed in Chapter 23.

Specifying a tftpd Directory

OpenBSD's `tftpd(8)` serves files from a directory, much like a web server. Traditionally, this directory is */tftpboot*, but don't follow tradition in this case (you don't want a TFTP user filling your server's root partition!). If you use */tftpboot* on your root partition, make sure that your TFTP clients can't write to the directory. (You could create a */tftpboot* partition.) Normally, I create */var/tftpboot* and tell `tftpd` to use that as its root directory. If your fingers are used to typing `/tftpboot`, create a symlink.

To enable `tftpd`, set `tftpd_flags` in *rc.conf.local* to the TFTP root directory.

```
tftpd_flags="/var/tftpboot"
```

`tftpd` chroots to the directory you specify, so `tftpd` cannot access files outside this directory.

tftpd and Files

TFTP uses file permissions as an access control method. Because all files on the TFTP server can be read by anyone who can access the server port, TFTP will let clients read files in its root directory only if they are world-readable. To make them world-readable, do this:

```
# chmod +r /var/tftpboot/filename
```

Similarly, `tftpd` will not allow anyone to upload a file unless a file of that name already exists and is world-writable. This means that anyone who knows a file's name can overwrite it, so make vital files read-only. If an attacker can't write files, he can't fill your hard drive.

To create files via TFTP, so that you can upload files that don't already exist, run `tftpd` with the `-c` option.

`tftpd` starts as root in order to bind UDP port 69, but it then drops privileges and runs as the unprivileged user `_tftpd`. Any files `tftpd` created will be owned by its user. As a general rule, the files in the TFTP root directory should not be owned by `_tftpd` , in order to make sure that the server cannot affect the files it serves.

tftpd Logging

You should log your TFTP transfers. Use the `-v` flag to send the transaction log to `syslogd`.

```
tftpd_flags="-v /var/tftpboot"
```

`tftpd` logs uses the FTP facility to log messages to */var/log/daemon*.

Testing the TFTP Server

Use `tftp(1)` to test your TFTP server.

```
$ tftp caddis
tftp> get testboot.iso
Received 20879569 bytes in 10.4 seconds
```

You won't see any friendly hash marks as you download the file, and you can't change to another directory or list the contents of the TFTP server. Once the test is complete, use `quit` to end your TFTP session.

After you have a TFTP client and server set up, you'll be ready to serve diskless OpenBSD machines, router operating system images, or anything else you need.

The SNMP Agent snmpd

SNMP is the de facto standard for gathering information from network devices. Many different devices from many different vendors support SNMP as a management protocol.

OpenBSD includes an SNMP agent, `snmpd(8)`, which supports all of the usual SNMP functions, and also offers visibility into OpenBSD-specific features such as packet filtering.

SNMP works according to the standard client/server model. The SNMP client (usually a server performing network management or monitoring) queries the SNMP server (or *agent*) running on a network device. The SNMP agent, `snmpd`, gathers information from the local system and returns it to the client.

In traditional SNMP, an SNMP client with the correct privileges can also request that the SNMP agent modify its device. Most Unix-like operating systems are designed to be configured at the command line and generally don't accept write requests from SNMP. OpenBSD follows this trend, and we will focus specifically on read-only SNMP.

In addition to having an SNMP agent answer requests from an SNMP client, the agent can transmit SNMP traps to a *trap receiver* somewhere on the network. SNMP traps are much like `syslogd(8)` messages, except that they follow a specific format required by SNMP.

NOTE *OpenBSD does not include an SNMP trap receiver. If you need one, check out `snmptrapd` in the `net-snmp` package.*

SNMP MIBs

SNMP manages information via a Management Information Base (MIB), which is a tree-like structure that contains hierarchical information in ASN.1 format. Each SNMP agent has a list of information it can extract from the local system, arranged in a hierarchical SNMP MIB with very general main categories, such as network, physical, programs, and so on.

Think of the MIB tree as a well-organized filing cabinet, where individual drawers hold specific information, and files within drawers hold particular facts. Similarly, the uppermost MIB contains a list of MIBs beneath it.

MIB References

MIBs can be referred to by name or number. For example, here's a MIB pulled from an OpenBSD test machine:

```
interfaces.ifTable.ifEntry.ifDescr.1 = STRING: "em0"
```

The first term in this MIB, `interfaces`, tells us that we're looking at this machine's network interfaces. If this machine had no interfaces, this category would not even exist (although an OpenBSD machine will always